Report

Grigorii Guryeb

March 2026

1 Introduction

Although AdamW is currently the de facto standard and is used nearly universally, recent years have seen the emergence of promising alternatives such as Muon and MeZO. The Muon optimizer leverages matrix orthogonalization for weight updates—an approach that excels at handling large parameter matrices and significantly accelerates convergence. In contrast, MeZO is a state-of-the-art zeroth-order optimization method designed to drastically reduce memory overhead by bypassing backpropagation. This study aims to provide a comparative analysis of these strategies when fine-tuning small language models using LoRA on limited datasets.

2 Optimization algorithms

2.1 AdamW

The AdamW optimizer represents a significant refinement of the standard Adam algorithm, specifically addressing the suboptimal interaction between adaptive learning rates and L_2 regularization. While the original Adam optimizer incorporates L_2 regularization by adding the penalty term directly to the objective function $f(\theta)$, AdamW employs decoupled weight decay. In this framework, the weight update is modified by subtracting a decay term proportional to the parameter value directly from the current weights, rather than merging it into the gradient-based first and second moment estimates (m_t and v_t).

2.1.1 Algorithm

$g_t = \nabla_\theta f_t(\theta_{t-1})$
$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$
$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$
$\hat{m}_t = \frac{m_t}{1-\beta_1^t}$
$\hat{v}_t = \frac{v_t}{1-\beta_2^t}$
$\theta_t = \theta_{t-1} - \eta_t(\alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t}+\epsilon} + \lambda \theta_{t-1})$
where η_t is a learning rate on step t and $\lambda, \beta_1, \beta_2$ is a parameters.

2.2 Muon

Muon is a matrix-aware optimization algorithm that applies spectral orthogonalization to gradient updates using the Newton–Schulz iteration.

2.2.1 Algorithm

$g_t = \nabla_\theta f_t(\theta_{t-1})$
$B_t = \mu B_{t-1} + g_t$
$O_t = NewtonShulz5(B_t)$
$\theta_t = \theta_{t-1} - \eta O_t$
where η - learning rate and μ is momentum

2.3 Mezo

MeZO is zeroth-order algorithm using only two forward passes to approximate gradient. The gradient estimation in MeZO is grounded in a variational approximation of the directional derivative. For a target objective function $J(\theta)$ and a random perturbation vector $z \sim \mathcal{N}(0, \mathbf{I})$, the true gradient $\nabla J(\theta)$ is approximated via a finite-difference quotient in the direction of z:

$$\hat{g} = \frac{J(\theta + \epsilon z) - J(\theta - \epsilon z)}{2\epsilon} z$$

where ϵ denotes the perturbation scale, a small scalar coefficient. A critical implementation feature is the utilization of a deterministic pseudo-random number generator: the vector z is reconstructed on-the-fly using a fixed seed value. This approach effectively eliminates the requirement to store the high-dimensional perturbation tensor in GPU memory, facilitating significantly higher memory efficiency.

2.3.1 Algorithm

$\theta = PerturbParameters(\theta, \epsilon, s)$
$l_+ = f(\theta, B)$
$\theta = PerturbParameters(\theta, -2\epsilon, s)$
$l_- = f(\theta, B)$
$\theta = PerturbParameters(\theta, \epsilon, s)$
$Projectedgrad = \frac{l_+ - l_-}{2\epsilon}$
for $\theta_i \in \theta$ do
$z \sim N(0, 1)$
$\theta_i = \theta_i - \eta_t \cdot Projectedgrad \cdot z$
where $PerturbParameters(\theta, \epsilon, s)$ is
for $\theta_i \in \theta$ do
$z \sim N(0, 1)$
$\theta_i \text{ ¸} \theta_i + \epsilon z$

3 Setup

3.1 Model: Qwen2.5-0.5B

About 0.5 Billion parameters decoder-only transformer model.

3.2 LoRA configuration

rank r: 32
lora alpha: 32
target modules: q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
dropout: 5%
bias: none

3.3 HyperParameters

Base Learning rate:

Optimizer	Learning Rate
Baseline	0.695
AdamW	1e-4
Muon	0.005
Hybrid	Adamw: 0.0001— Muon: 0.001
MeZO	1e-7

Table 1: PIQA evaluation results

Epochs: 4
Batchs size: 32

3.4 Dataset: OpenWebText-100k

Used about 6% of dataset (6000 examples)
Sequence length: 2048

3.5 Hardware

GPU:NVIDIA GeForce RTX 3060 (locked on 1500 frequency and 100 wt power usage)
CPU: AMD Ryzen 5 5600X
RAM: 32 GB
VRAM: 12 GB

4 Training Dynamic and Results

4.1 Training dynamic

None of the optimizers yielded a reduction in loss during training; notably, MeZO exhibited a divergent error trend. Hybrid and AdamW recorded the lowest loss levels, marginally outperforming Muon. The peak loss observed in MeZO is inherent to its specific architecture.

Figure 1: Loss dynamic

The methods exhibited varying levels of VRAM utilization. Muon had the highest memory consumption at 6362 MB. Hybrid and AdamW used nearly identical amounts (5456 MB and 5,486 MB, respectively), while MeZO was the most efficient, consuming only 4030 MB.

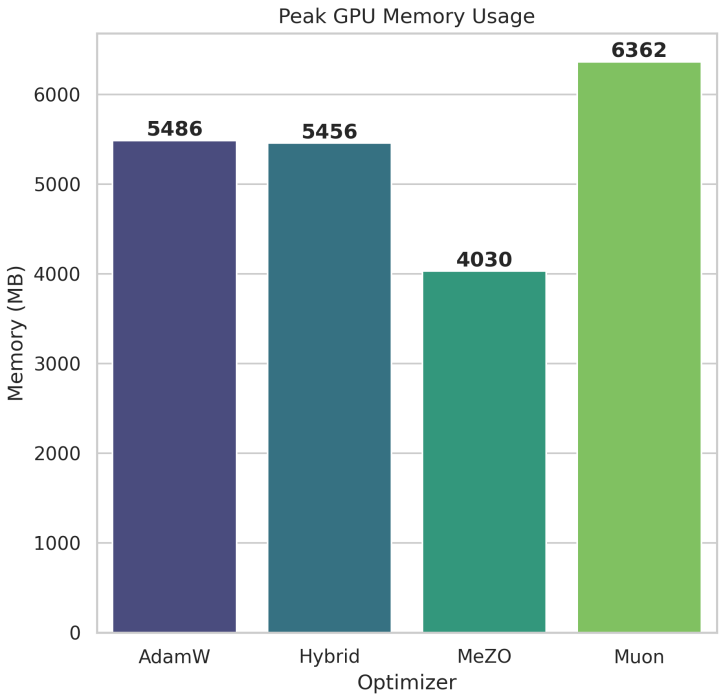

Figure 2: Memory usage

Training time per step also varied across the optimizers. AdamW and Hybrid performed similarly, ranging between 0.8–0.9 seconds per iteration. Vanilla Muon performed slightly better (0.75–0.85 seconds), while MeZO was the fastest, completing iterations in 0.6–0.65 seconds.

4.2 Training results

Model validation was conducted using the PIQA (Physical Interaction QA) dataset. The baseline Qwen2.5-0.5B model achieved an acc_norm of approximately 0.695. The validation results for each optimization method on PIQA are as follows:

Optimizer	PIQA acc_norm
Baseline	0.695
AdamW	0.6991
Muon	0.5800
Hybrid	0.7013
MeZO	0.7008

Table 2: PIQA evaluation results

Among the tested methods, only Muon showed a performance degradation. While the remaining methods surpassed the baseline normalized accuracy, the gains were marginal. Consequently, these results do not suggest a statistically significant improvement over the original model.

5 Conclusion

None of the methods yielded a significant improvement in model performance. These modest results are primarily attributed to the limited dataset size (6,000 samples) and the use of LoRA for fine-tuning such a small model.

Hybrid optimization achieved the best downstream performance,while MeZO demonstrated the best resource efficiency. Furthermore, the results indicate that Muon performs poorly when paired with LoRA; since the matrices it attempts to orthogonalize are relatively small, standard AdamW proves more effective. Combined with its higher VRAM consumption, MMuon showed degraded performance in the LoRA fine-tuning setting. This may be explained by the relatively small parameter matrices introduced by LoRA, which limit the benefits of Muon's matrix-level orthogonalization.